



UNIVERSITY OF PORTSMOUTH

Coursework for Programming for Data Analytics and AI 2024/25

**School of Computing
University of Portsmouth**

MODULE CODE: M33145

**COORDINATOR: Dr ATEFEH KHAZAEI
GHOOZHDI**

Student Id: 2302988

Table of Contents

1. Introduction:	3
2. Descriptive Analytic.....	3
a. Basic statistics for numerical columns:	3

b. Basic statistics for categorical columns.	4
c. Visualization:.....	5
i. Pie Chart Showing the Distribution of Stroke Data in the Dataset:	5
ii. Density Chart:	6
iii. Stacked Bar Chart:	7
iv. Bar Chart:	8
v. Violin Chart:	9
3. Data Preparation:	9
a. Dropping columns:.....	10
b. Handling Missing Values:	11
c. Outlier:.....	11
d. Discretization:	12
e. Oversampling:	13
f. Normalization:	13
g. Encoding:	13
4. Classification	14
a. SVM:	14
b. Random forest:	15
c. MLP classifier	15
d. XGboost:	16
5. Regression	18
a. Linear Regression:	18
b. Catboost Regression:.....	18
6. Clustering:	19
a. K-means :.....	19
b. Gaussian Mixture Model:	23
Comparison:.....	25
7.Reference:	25

1. Introduction:

The aim of this report is to apply a variety of analytical tools to analyse the stroke dataset and obtain significant findings. Descriptive analysis, data visualization, stroke column classification, bmi regression, and clustering on the provided data are all part of the task. To fulfil the goals of this analysis, the outcomes of each method will be shown, explained, and evaluated.

2. Descriptive Analytic

Before basis statics data was imported in Google Colab from Google Drive and data was assigned to dataframe df.

a. Basic statistics for numerical columns:

The dataset includes both numerical and categorical data describing various demographic, health, and lifestyle features. The numerical statistics show notable variation in age, glucose levels, and BMI, while the categorical data highlights dominant categories in each feature.

Columns	Count	Mean	Std	min	25%	50%	75%	max
age	5110.00	43.226	22.612	0.080	25.000	45.000	61.000	82.000
hypertension	5110.00	0.097	0.296	0.000	0.000	0.000	0.000	1.000
heart_disease_s	5110.00	0.054	0.226	0.000	0.000	0.000	0.000	1.000
avg_glucose_level	5110.00	106.147	45.283	55.120	77.245	91.885	114.090	271.740
bmi	4909.00	28.893	7.854	10.300	23.500	28.100	33.100	97.600
stroke	5110.00	0.048	0.215	0.000	0.000	0.000	0.000	1.000

- **age:** The dataset contains 5110 records with an average age of 43.23 years, a standard deviation of 22.612, and ages ranging from 0.08 to 82 years. The 25th, 50th, and 75th percentiles are 25, 45, and 61 years, respectively.
- **hypertension:** The average value is 0.097 which indicates that only around 9.7% of individuals have hypertension.

- **Heart_Disease:** The mean value is 0.054 which shows that approximately 5.4% of individuals have heart disease.
- **Avg_Glucose_Level:** The average glucose level is 106.147 mg/dL having a standard deviation of 45.283. Values range from 55.12 to 271.74 mg/dL, and the interquartile range spans 77.245 to 114.09 mg/dL.
- **bmi:** The mean of bmi is 28.893 with a standard deviation of 7.854. BMI ranges from 10.3 to 97.6, with percentiles at 23.5 (25%), 28.1 (50%), and 33.1 (75%).
- **stroke:** The mean of stroke value is 0.048 which indicates that 4.8% of individuals experienced a stroke.

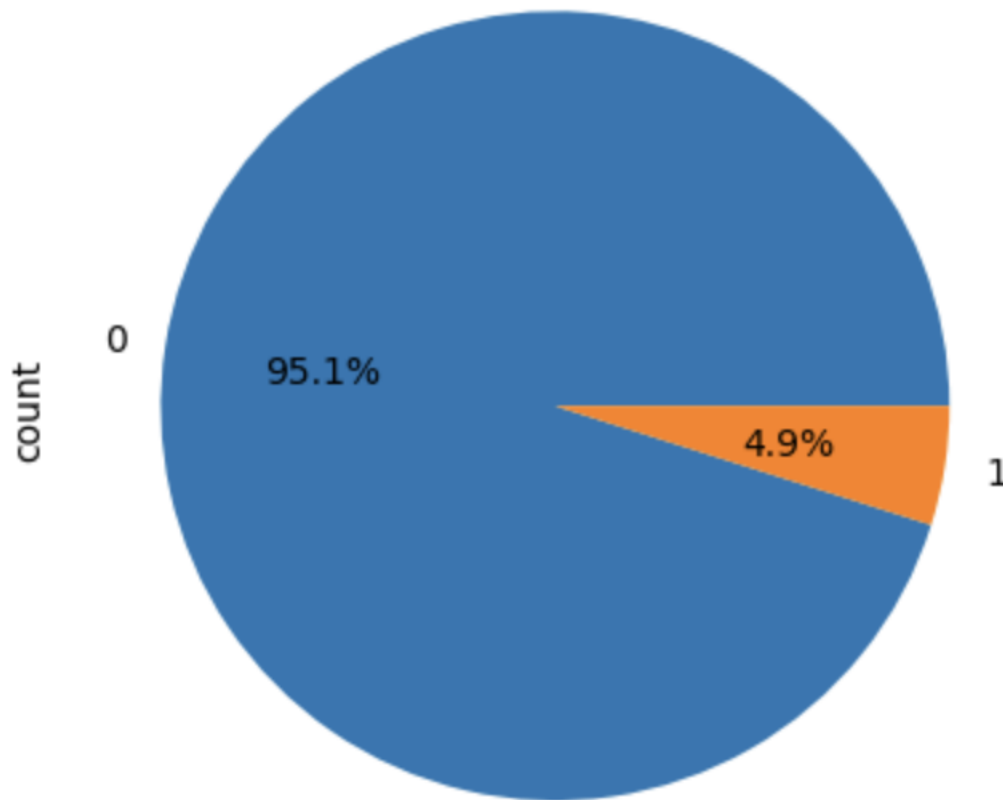
b. Basic statistics for categorical columns.

Columns	Count	Unique	Top	Freq
gender	5110.00	3	female	2994
ever_married	5110.00	2	yes	3353
work_type	5110.00	5	private	2925
Residence_type	5110.00	2	urban	2596
Smoking_status	5110.00	4	Never smoked	2892

- **Gender:** There are 3 unique categories, 'female' is the most frequent with 2994 occurrences.
- **Ever Married:** Two categories are present, with 'yes' being the most common (3353 instances).
- **Work Type:** This includes 5 categories, with 'private' being the most frequent (2925 occurrences).
- **Residence Type:** There are two varieties, with 'urban' being slightly more common (2596 records).
- **Smoking Status:** There are four distinct categories, the most common being 'never smoked' (2892 occurrences).

c. Visualization:

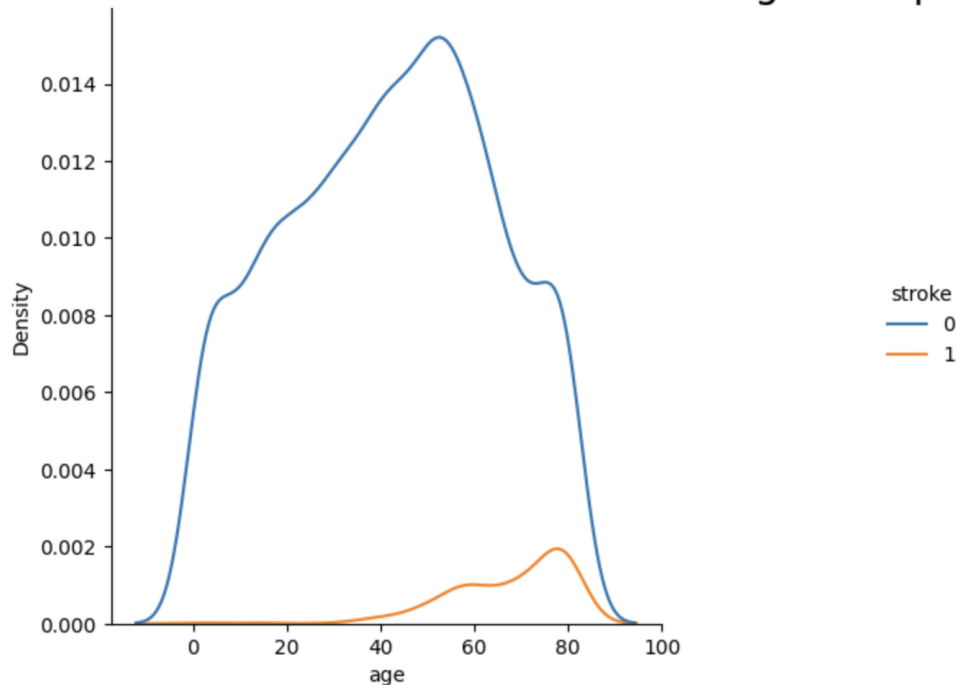
i. Pie Chart Showing the Distribution of Stroke Data in the Dataset:



As seen in the visual above, there were 4.9% of people who had a stroke (1) and 95.1% who did not (0). This means a relatively small proportion of the dataset sample have a stroke whilst the other parameter samples do not have a stroke. The dataset is highly unbalanced with the "no stroke" category prevalent, which might affect the performance so, to improve machine learning performance it requires adapting the weights of the model to ensure balanced analysis.

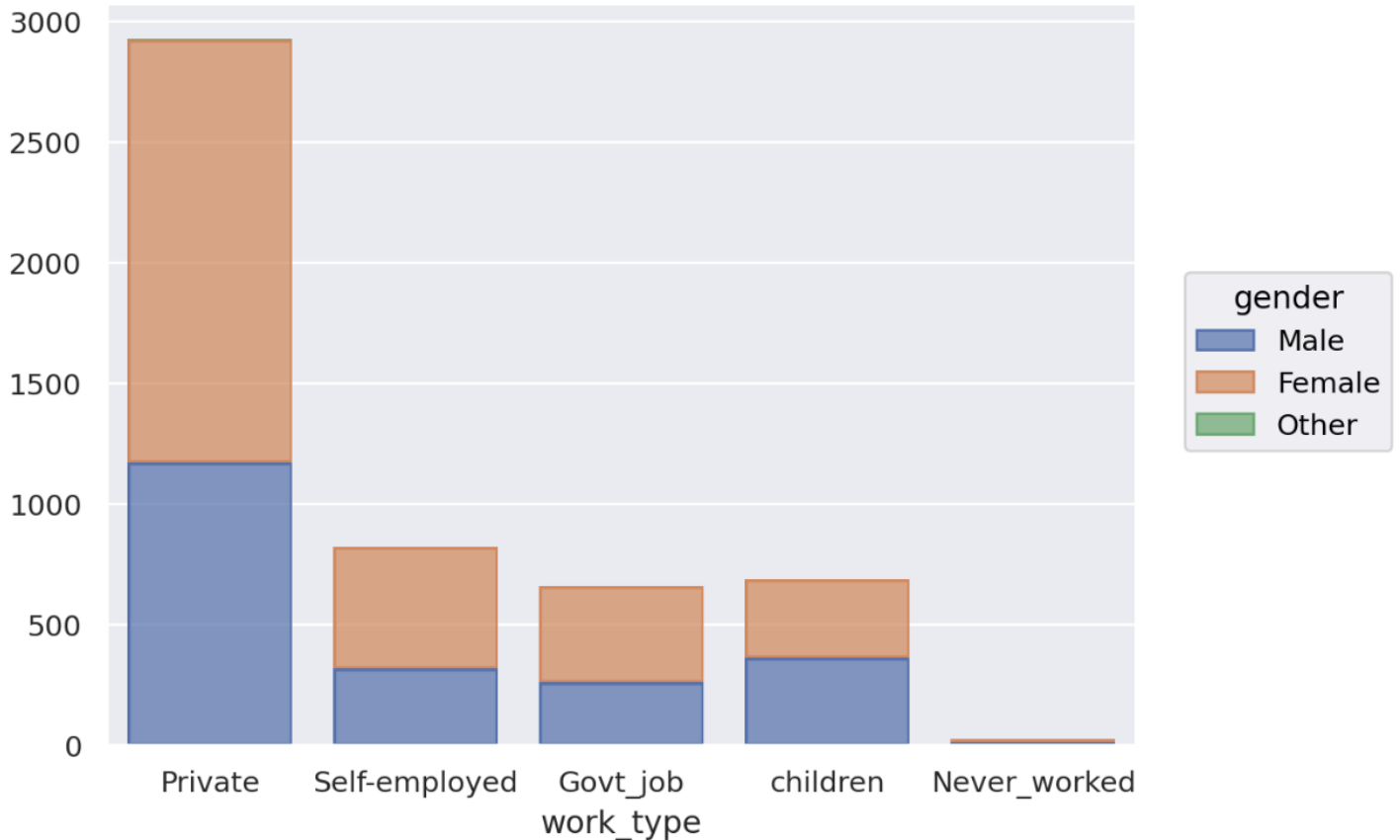
ii. Density Chart:

Exploring Stroke Occurrence Across Different Age Groups



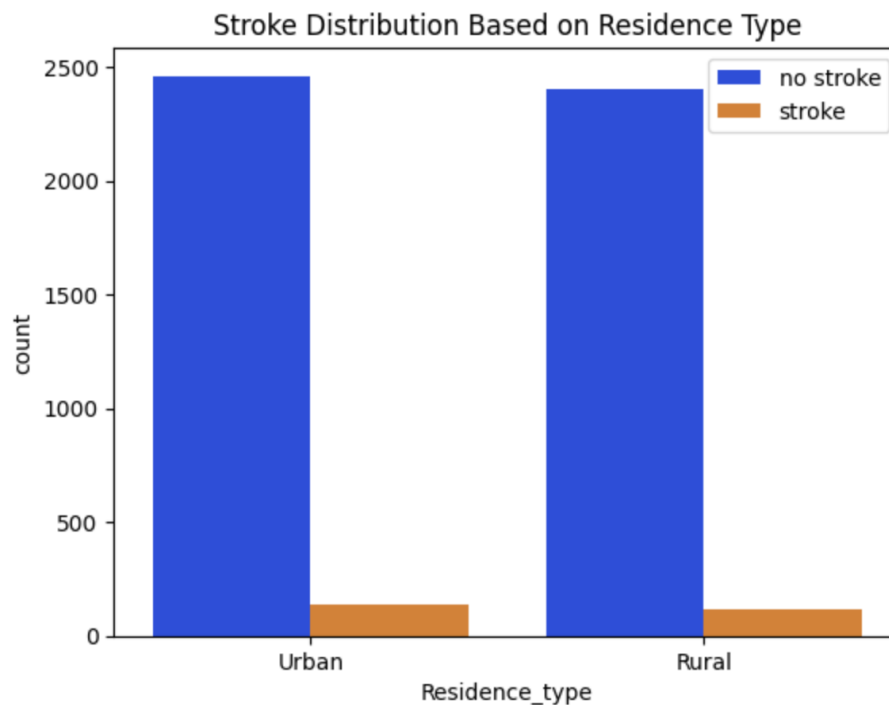
This graph shows the frequency of strokes in various age groups. The density plot shows the distribution of people who had no stroke (stroke = 0) against those who had one (stroke = 1). People without strokes are represented by the blue curve, which has a high density in the middle-aged group, especially between the ages of 40 and 60, and a diminishing density as people age. The orange curve, which shows stroke victims, has a lower density overall but peaks in older age groups, especially beyond age 60, suggesting that older people are more likely to have strokes. This is a definite age-related trend which means that as people age, their risk of stroke rises.

iii. Stacked Bar Chart:



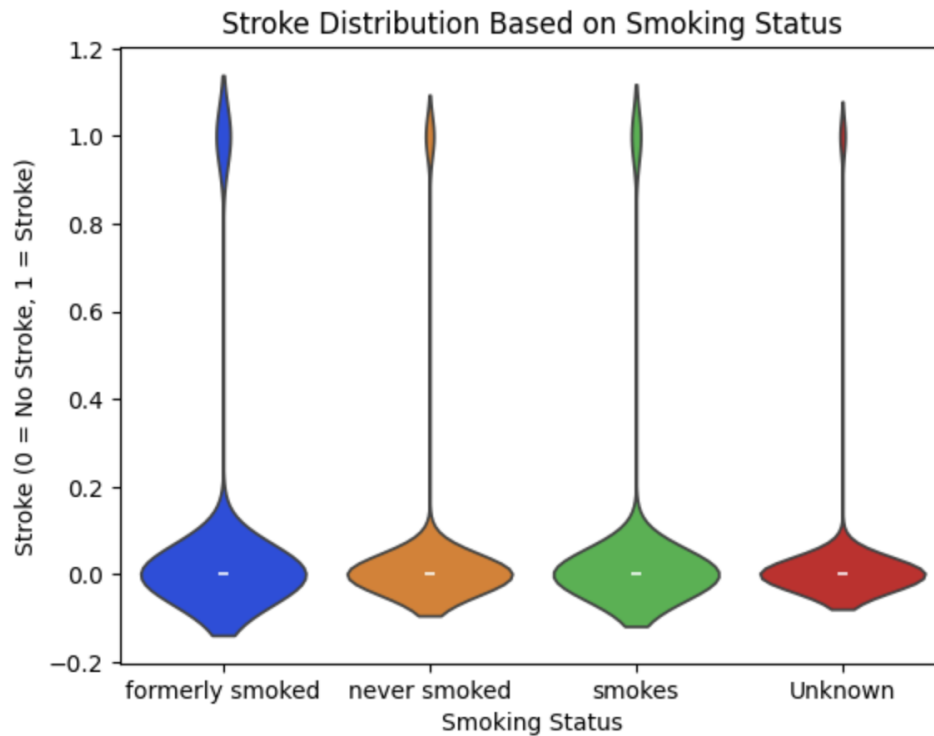
The distribution of people by gender (Male, Female, and Other) and various work kinds (Private, Self-employed, Govt_job, children, and Never_worked) is displayed in the bar chart. Females significantly outnumber males, and most people work in private employment. Nearly equal gender proportions are found in the government and self-employed job groups, which have moderate presence. The 'Never worked' category has the fewest number, perhaps consisting of retirees or those without a work history, while the 'Children' category has fewer people, representing those without a formal employment type. The predominance of private employment and the balanced representation of gender in the majority of categories are highlighted in this graphic.

iv. Bar Chart:



The distribution of strokes by type of residence (rural versus urban) and smoking status is compared in a graph. The blue bars indicate that most people in both urban and rural locations did not have a stroke, whereas the orange bars indicate that a small percentage of people had a stroke. Between urban and rural locations, the distribution of strokes seems to be fairly balanced. The 'no stroke' category (blue) once again dominates across all groups, whereas the 'stroke' category (orange) makes up a significantly smaller percentage, irrespective of smoking status, according to the lower graphic, which looks at the distribution of strokes by smoking status. This indicates that while stroke rates are low for both types of residences and smoking statuses, stroke risk may be influenced by other factors. most categories.

v. Violin Chart:

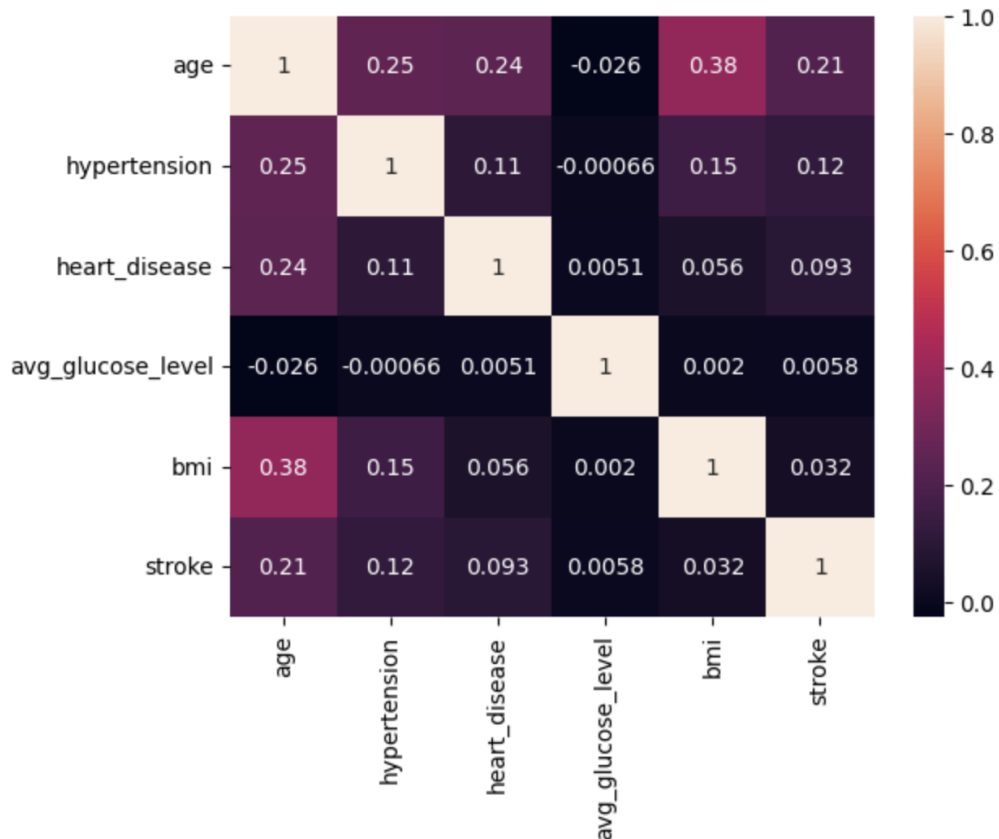


Each of the following smoking statuses are indicated on the chart: 'formerly smoked,' 'never smoked,' 'smokes,' and 'unknown.' A stroke is tagged as 1 and no stroke is tagged as 0. Violin plots show the frequency (y-axis) and distribution of stroke outcomes (x-axis) within each category, as well as the density of the events. The image shows that the majority of the data points are around 0 (no stroke) across all categories, indicating that strokes are less prevalent overall regardless of smoking status. But there is a slight variation in the plot widths and plot shapes, suggesting that the stroke probabilities differ for these groups. For example, the spreads for 'smokes' and 'formerly smoked' are slightly wider around 1 (stroke)—which might imply a stronger association with stroke than for 'never smoked.' (Characteristics in the 'unknown' group resemble those of the other categories, but they are less pronounced due to a lack of data or fluctuation.

3. Data Preparation:

a. Dropping columns:

As the id column is not useful so this column is dropped.



The image shows a heatmap of correlation coefficients between different variables in a dataset. Here's an analysis and suggestions for dropping columns:

- age: Moderate positive correlation 0.21. Keeping this variable as it may provide meaningful insights into predicting strokes.
- hypertension: Weak positive correlation 0.12.
- heart_disease: Very weak positive correlation 0.093. Consider keeping this, as it might still provide useful information, especially with additional features.

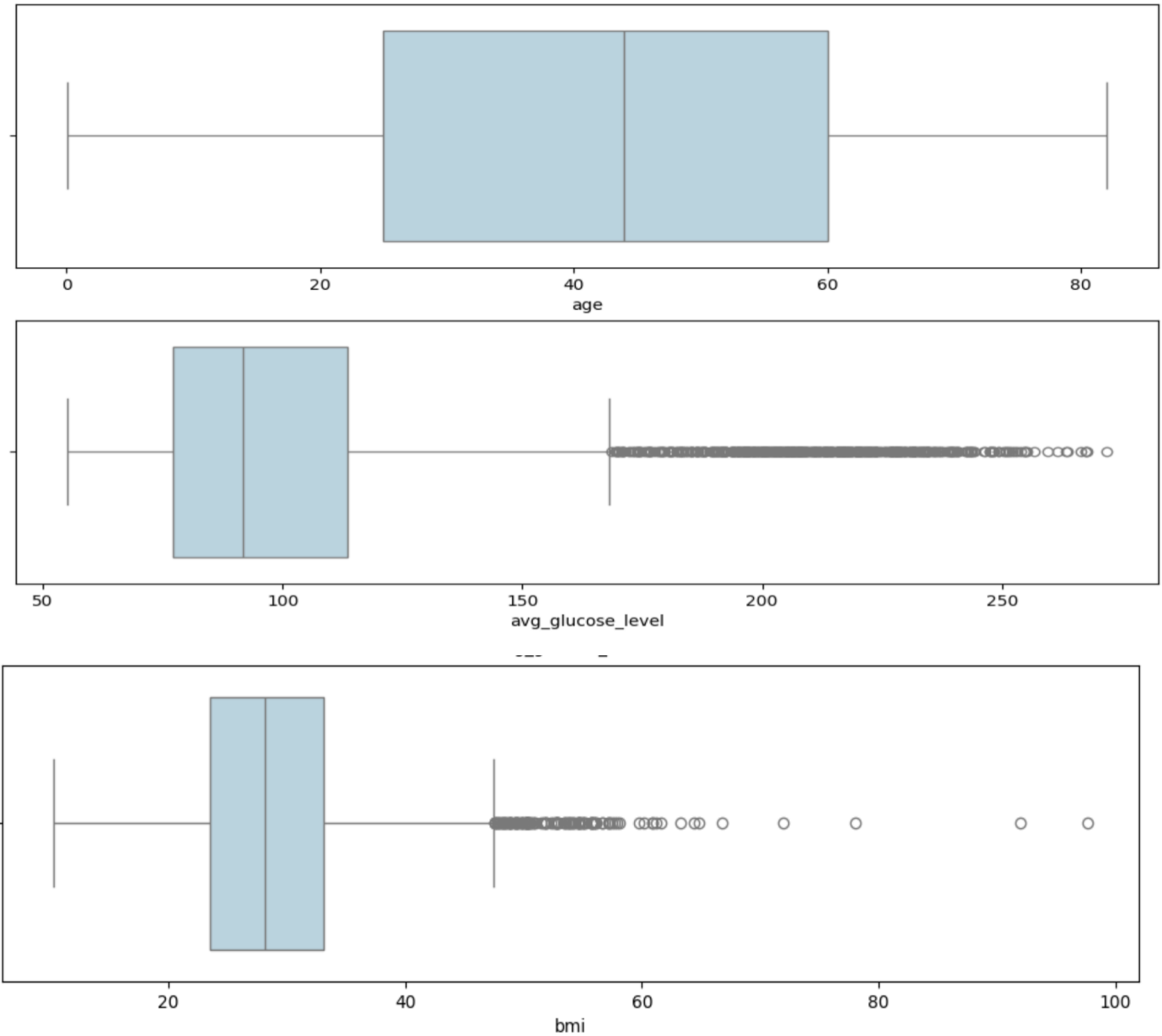
- `avg_glucose_level`: Very weak positive correlation (0.0058). While its correlation is low, domain knowledge suggests it is likely relevant for stroke prediction. Keeping it for further modeling.
- `bmi`: Very weak positive correlation (0.032). Since `bmi` is generally considered a health indicator, I should keep it for further analysis.

b. Handling Missing Values:

There are 201 missing values in the column `bmi` which is just 3.933% of the total data. It is removed from the data as it is very less and may not affect the characteristics of the data.

c. Outlier:

A boxplot was plotted to check for outliers in the numerical variables in the dataset and output is shown below. As from the figures below, `avg_glucose_level` and `bmi` contains outlier which is removed using interquartile range (IQR) method. The function `remove_outliers` by calculating Q1 and Q3 for `avg_glucose_level` and `bmi` column in a DataFrame and determining the IQR. Data points which fall below $Q1 - 1.5 * IQR$ or above $Q3 + 1.5 * IQR$ were removed. This function is applied to the columns '`avg_glucose_level`' and '`bmi`' to remove outliers. This method ensures that outlier values are excluded, helping to improve the quality of analysis by focusing on non outlier data points.

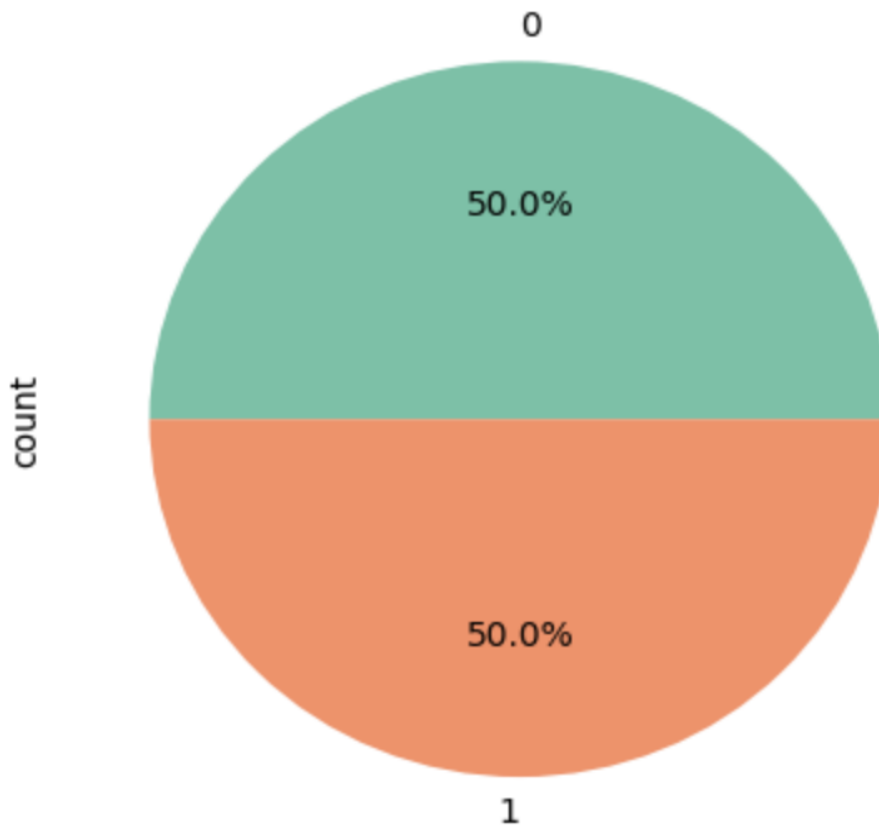


d. Discretization:

The discretization process is applied to categorize continuous numerical values in the column (age) of the DataFrame `df` into predefined age groups. The age is compared against specified ranges: 'Infant' for ages 0 to 2, 'Child' for ages 3 to 12, 'Adolescent' for ages 13 to 18, 'Young Adults' for ages 19 to 35, 'Middle Aged Adults' for ages 36 to 60, and 'Old Aged Adults' for ages above 60. Based on these conditions, each value in the age column is converted into an appropriate category in the new `age_group` column.

e. Oversampling:

In this case, oversampling is used to balance an imbalanced dataset equalizing the number of instances in the 2 classes of the stroke column (1 for had a stroke and 0 for no stroke). First, datasets is divided into two sets called: `had_stroke` (where stroke is 1) and `no_stroke` (where stroke is 0). Next, the `resample ()` is called on rows of `has_stroke`, which generates a sample of the same number of rows as `no_stroke` by randomly duplicating any given row in `has_stroke`. Lastly, the upsampled `had_stroke` is merged with `no_stroke` to make a new balanced dataset called `upsampled_data`. By doing so, we can solve the class imbalance problem in classification.



f. Normalization:

Min-Max Scaling is applied to the columns 'age', 'avg__glucose__level', and 'bmi' in the `model_data` DataFrame which are numerical columns to bring the data values to a common scale, making them easier to interpret.

g. Encoding:

LabelEncoder is used for converting categorical data to numerical of `age_group`, `gender`, `hypertension`, `heart_diseases`, `ever_married`, `work_type`, `Residence_type` and `smoking_status` columns of the `model_data` DataFrame.

4. Classification

For classification purposes, the dataset is separated into training datasets and testing datasets. 20% of the data is separated for testing, while 80% of data is used for training. Then the resulting splits are stored in X_train, X_test, y_train, and y_test, which is used for training and testing a machine learning model.

a. SVM:

I used the SVM model with linear, model2 with RBF (Radial Basis Function), and model3 with sigmoid kernels for the classification task.

Performance metrics:

Model	Confusion Matrix	Accuracy	Precision	Recall	F1-score	AUC
SVM1	[[595,246] [154, 652]]	0.75	0.76	0.76	0.76	0.75
SVM2	[[537 304] [98 708]]	0.75	0.77	0.76	0.75	0.75
SVM3	[[590 251] [215 591]]	0.71	0.72	0.72	0.72	0.71

This chart presents performance metrics for three Support Vector Machine (SVM) models (SVM1, SVM2, and SVM3) for a classification task. Confusion matrix with respect to SVM1 shows 595 true positives and 652 true negatives with 246 false positives and 154 false negatives signaling equal rate of predictions (true and false). The results indicate a slight improvement in SVM2, with the model producing 537 true positives and 708 true negatives, but also 304 false positives and 98 false negatives simultaneously, supporting the notion that the model is better at correctly assessing true negatives. Conversely, SVM3 shows the lowest performance with 590 true positives and 591 true negatives but more false positives (251) and false negatives (215), indicating more errors of classification than the other two models. From these matrices, we notice that SVM2 is the best model for overcoming misclassifications; the SVM3 is the worst one.

Some important metrics are: confusion matrix, accuracy, precision, recall, F1-score, AUC (Area Under Curve). From the above table, we see that, among the models used, SVM2 has the best performance with 0.77 of accuracy, 0.77 of precision, 0.78 of recall, 0.77 of F1-score and 0.77 of AUC respectively, which demonstrates that SVM model is robust in dealing with different styles of data including complementary data. SVM1 is not much better than random — with an accuracy of 0.75, and precision (0.76) and recall (0.76) proportions roughly one to one. SVM3 is the least performing in all metrics, achieving accuracy, precision, recall, F1-score, and AUC of 0.71, 0.72, 0.72, 0.72, and 0.71 respectively which indicates its ineffectiveness for such type of classification task.

b. Random forest:

I used three Random Forest models for the classification task with different configurations: the first model had a 10 maximum depth , the Gini impurity criterion, and a random state set to 0; the second model had a maximum depth of 10, 150 estimators, the entropy criterion, and a random state set to 0; and the third model had a maximum depth of 5, 100 estimators, the entropy criterion, and a random state set to 0.

Performance metrics:

Model	Confusion Matrix	Accuracy	Precision	Recall	F1-score	AUC
RF1	[[773 68] [0 806]]	0.95	0.96	0.96	0.96	0.95
RF2	[[772 69] [0 806]]	0.95	0.96	0.96	0.96	0.95
RF3	[[555 286] [21 785]]	0.81	0.82	0.81	0.81	0.81

The RF1 results such as: 773 true positive, and 806 true negative and only 68 false positive and 0 false negative show good classification ability. Again, RF2 has practically the same numbers with 772 true positives, 806 true negatives, 69 false positives, and no false negatives, so it has similar performance as RF1. In contrast, while RF3 has fewer true positive and true negative (555,785) pairs performance, it had the highest misclassification of images with 286 those that belonged to other classes, as well as 21 false negatives. So, to summarize the results: RF1 and RF2 are equally robust and very accurate for the given classification task, while RF3 is worse because it misclassifies observations considerably more often than the previous two.

Common metrics of three Random Forest models (RF1, RF2, and RF3) for a classification task. RF1 and RF2 achieved the same and best performance among all metrics as shown by an accuracy of 0.95, precision of 0.96, and recall of 0.96 (RF1) or 0.97 (RF2), F1-score of 0.96, and AUC at 0.95, indicative of excellent capabilities for classification. For RF3, the accuracy is 0.81, precision is 0.82, recall is 0.81, F1-score is 0.81, AUC is 0.81, which is moderate compared with RF1 and RF2. The RF1 and RF2 show outstanding performance in this classification task, and the RF3 performs relatively low.

c. MLP classifier

I used three MLP (Multilayer Perceptron) models for the classification task with different configurations: the first model used the Relu activation function, Adam solver, a random state of 42, a maximum of 1000 iterations, and a single hidden layer with 100 neurons; the second model used the Relu activation

function, Adam solver, a random state of 42, a maximum of 1000 iterations, and three hidden layers with 100, 50, and 25 neurons, respectively; and the third model used the tanh activation function, SGD solver, a random state of 42, a maximum of 1000 iterations, and two hidden layers with 50 neurons each.

Performance metrics:

Model	Confusion Matrix	Accuracy	Precision	Recall	F1-score	AUC
MLP1	[[612 229] [101 705]]	0.79	0.81	0.80	0.80	0.80
MLP2	[[720 121] [24 782]]	0.91	0.92	0.91	0.91	0.91
MLP3	[[547 294] [99 707]]	0.76	0.78	0.76	0.76	0.76

As shown on table, MLP1 gives 612 true positives, 705 true negatives, 229 false negatives and 101 false positives, resulting in moderate accuracy (0.79) and precision (0.81) of the model. This means that, for the best performing model (MLP2) 720 of 841 predictions are true positive, 121 are true negative, meanwhile there are only 24 false positives and 101 false negatives leading to the highest accuracy (0.91), precision (0.92) and recall (0.91). In contrast, MLP3 yields the worst performance, with 547 true positives and 707 true negatives, but 294 false positives and 99 false negatives, producing lower accuracy (0.76), precision (0.78), and recall (0.76). The lower rates of misclassification with MLP2 compared to MLP3 displayed further confirmations to MLP2 being the most effective model since MLP3 has higher rates of misclassification.

Three MLP classifiers (MLP1, MLP2, MLP3) test the performance for classification task. The performance score of MLP2 shows an accuracy of 0.96, precision of 0.97, recall of 0.96, F1-score of 0.96, and an AUC of 0.96 (table 2), which shows that MLP2 has an excellent capability for classification. It comes very next, with accuracy 0.91, Precision of 0.92, Recall 0.91, F1-score 0.91, and AUC 0.91. MLP3 demonstrated the least performance, with 0.76 accuracy, 0.78 precision, 0.76 recall, 0.76 F1-score, and 0.76 AUC, thus effective with moderate performance. MLP2 overall achieves the best performance out of the three.

d. XGboost:

I used three XGBoost (Extreme Gradient Boosting) models for the classification task with different configurations: the first model used the default configuration; the second model had a maximum depth of 10, 200 estimators, and a learning rate of 0.1; and the third model had a maximum depth of 6, 100 estimators, and a learning rate of 0.05.

Performance metrics:

Model	Confusion Matrix	Accuracy	Precision	Recall	F1-score	AUC
XGB1	[[815 26] [0 806]]	0.98	0.98	0.98	0.98	0.98
XGB2	[[822 19] [0 806]]	0.98	0.99	0.99	0.99	0.98
XGB3	[[735 106] [5 801]]	0.93	0.94	0.93	0.93	0.93

With only 26 false negatives and no false positives, XGB1 reaches metrics of accuracy=0.98, precision=0.98 and recall=0.98 with 815 true positives and 806 true negatives. Where XGB2 has the highest performance of the previous models, with 822 true and 806 true negatives, and 19 and no false negatives and positives, respectively, which results in the best precision (0.99), recall (0.99) and F1-score (0.99). XGB3, on the other hand, does considerably worse with 735 true positives, 801 true negatives, 106 false negatives and 5 false positives, ending up with a slightly better (0.93) accuracy, precision (0.94) and recall score (0.93). These results strengthen the fact that XGB2 is the best model because of fewer misclassifications and XGB3 performs poorly as it has more false negatives and false positives.

Performance metrics for three XGBoost models (XGB1, XGB2, and XGB3) on a classification task. XGB1 and XGB2 have the best and very similar accuracy (0.98), precision (0.99), recall (0.99), F1-score (0.99), and AUC (0.98) with excellent classification capabilities. XGB3 came close, performing effectively with an accuracy of 0.93, precision of 0.94, recall of 0.93, F1-score of 0.93 and AUC of 0.93. These results show that XGB1 and XGB2 outperform in this task whereas the performance of XGB3 is moderately lower.

Comparison of models:

Different classifiers' performance was evaluated, namely SVM, Random Forest, MLP, and XGBoost. Among the models, the best performance was shown by the XGBoost (XGB2) model, with an (0.98; 0.99; 0.99; 0.99; 0.98) accuracy, precision, recall, F1-score, and AUC. This displays that it is robust and reliable for classification, as it regularly scored close to 100 percent across all performance metrics. A similar trend is seen in Random Forest (RF1/RF2) and MLP (MLP2) metrics as well, though their accuracy was around 0.95 and 0.96, respectively, with precision and recall values over 0.96. These outcomes imply that the Random Forest and MLP models are the very best tools for processing the given data set.

The SVM model (SVM2) had the lowest performance measuring accuracy (0.77), precision (0.77), recall (0.78), F1-score (0.77), and AUC (0.77) in comparison. Although these metrics show an acceptable performance, they are much lower than the other models. This discrepancy implies that either SVM fails to capture the dataset's complexity compared to the more advanced classifiers such as XGBoost and Random Forest. In summary, this makes XGBoost (XGB2) the best model for this classification task and SVM (SVM2) the worst model and should not be used if performance is desired.

5. Regression

For the regression the same preprocessed data is used (model_data). Then I trained the model with processed dataset, by splitting the features and target. We dropped the "bmi" column as this was the target variable we wanted to predict and so we had our independent variables. The data is splits into training and testing datasets. The ratio was 80% of data for training and 20% of data for testing to provide enough data to train the model while keeping some data for evaluation. For reproducibility purposes, a random state of 7 was chosen, and shuffling was performed before the split.

a. Linear Regression:

For the liner regression same preprocessed model_data is used. The "bmi" column is treated as our target variable. Then, the data was split into training and testing datasets using Scikit-learn. It is common and generally accepted to reserve most data, e.g. 80%, for training purposes, while holding 20% for testing, so that a model can be trained with adequate data yet not trained on the data that will be held back as test data.

Performance metrics:

models	MAE	MSE	RMSE	R2 Score
Linear Regression	4.68	34.33	5.85	0.10

This table presents evaluation of linear regression model with 4 metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE) and r2 score. The model MAE 4.68, which means the average result difference between predicted value and actual value is 4.68. The MSE, which captures average squared error, is 34.33, and the RMSE, that is square root of MSE, is 5.85. But the r2 score, a measure of how much of the target variable's variability the model can explain, is 0.10, indicating that the model explains nearly nothing of the variability of the target variable in this dataset. These metrics together suggest that the linear regression model does not fit well to the relationship between the features and the target variable.

b. Catboost Regression:

models	MAE	MSE	RMSE	R2 Score
--------	-----	-----	------	----------

Catboost Regression	0.07	0.01	0.11	0.52
------------------------	------	------	------	------

The summary of results of CatBoost regression model is the following table. It seems this model is more precise than Linear regression model with mean absolute error (MAE) of 0.07. When we look at the following data based on the Mean Squared Error (MSE), we can see that the MSE of 0.01 and the RMSE of 0.11 are both significantly lower than the previous linear regression model, which also implies a more precise Python model. CatBoost has an r^2 score of 0.52 which means that it explains 52% of the variance (better than the previous linear regression model). The results demonstrate that CatBoost achieves significantly better results that also better reflect the relation between the predictor variables and the target.

Comparison:

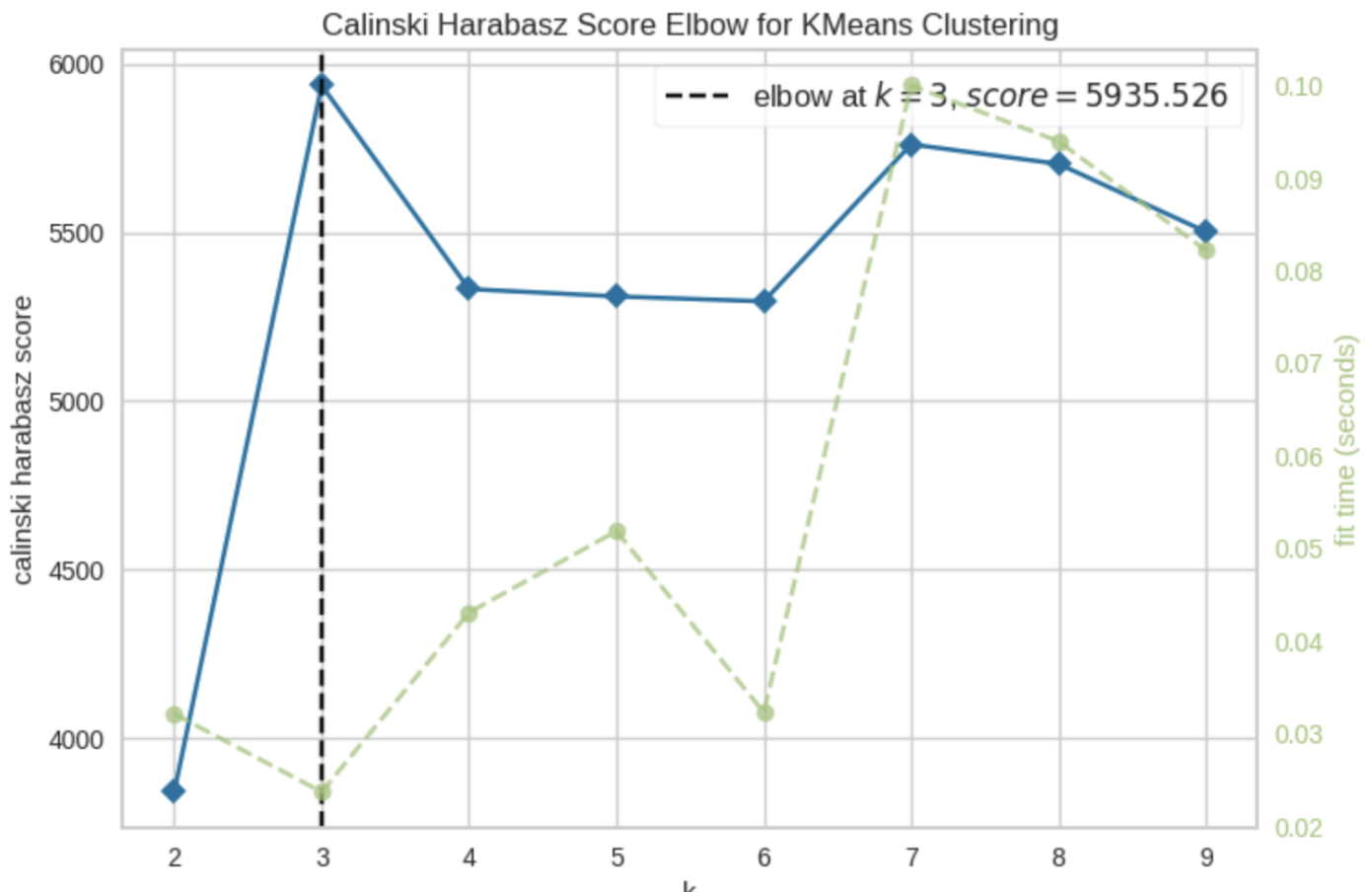
A comparison of the regression models shows that CatBoost outperformed Linear Regression for all evaluation metrics. The MAE, MSE, and RMSE of CatBoost were found to be 0.07, 0.01, and 0.11 respectively, which is much lower than Linear Regression's MAE of 4.68, MSE of 34.33, and RMSE of 5.85. Based on the r^2 score, CatBoost clearly outperformed with a score of 0.52, meaning it successfully accounted for 52% of the spread in the target variable, whereas Linear Regression only got an r^2 of 0.10, suggesting a weak performance. As observed from these results, CatBoost is the best-performing model as expected, meaning it's both accurate and captures the relationships present in the data well. In contrast, Linear Regression gave the least performance, as it was unable to capture complexities and nonlinear relationships existing in the data.

6. Clustering:

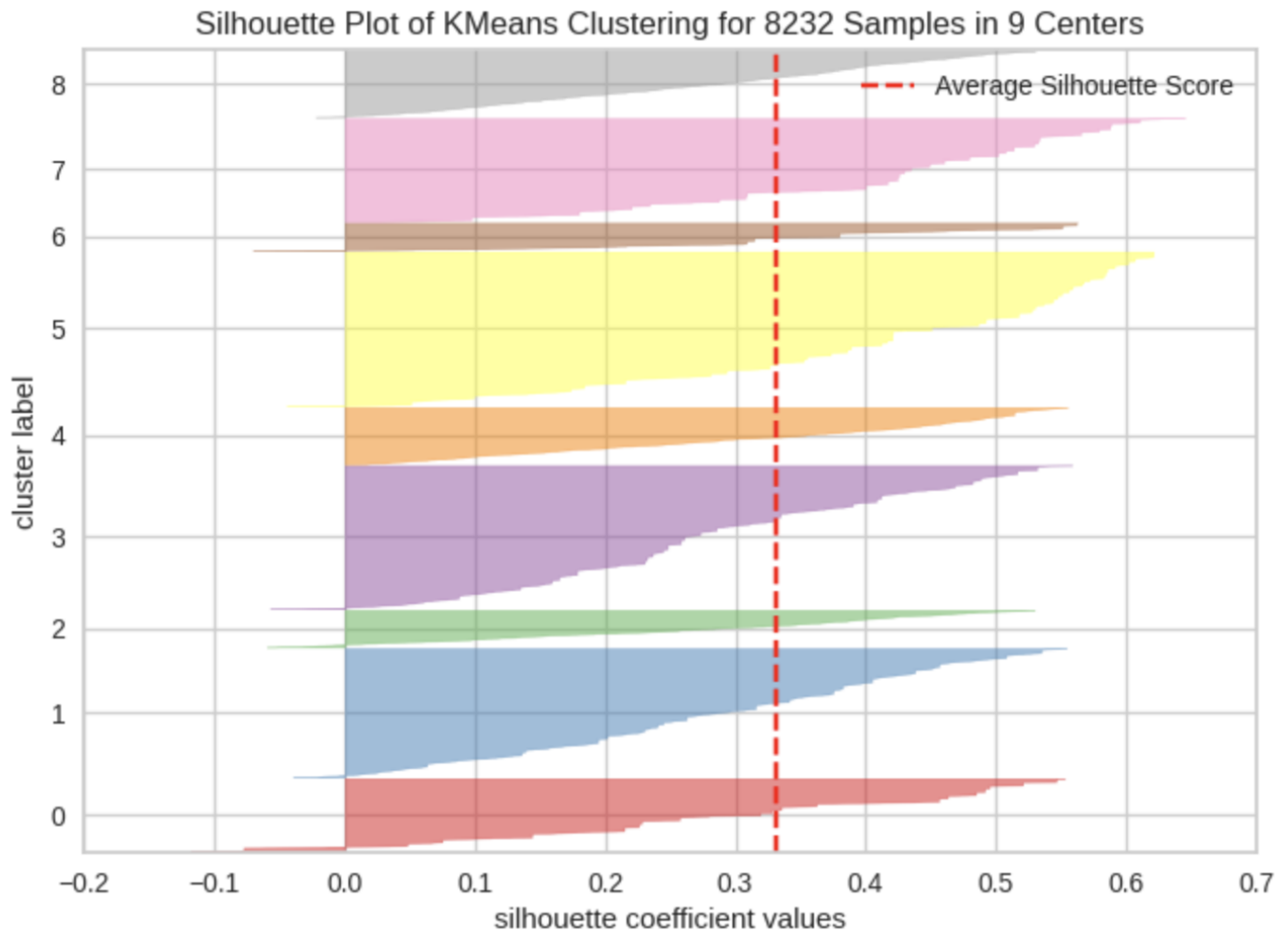
For clustering, I have used k – means clustering and Gaussian Mixture Model (GMM) and the same pre-processed data is used.

a. K-means :

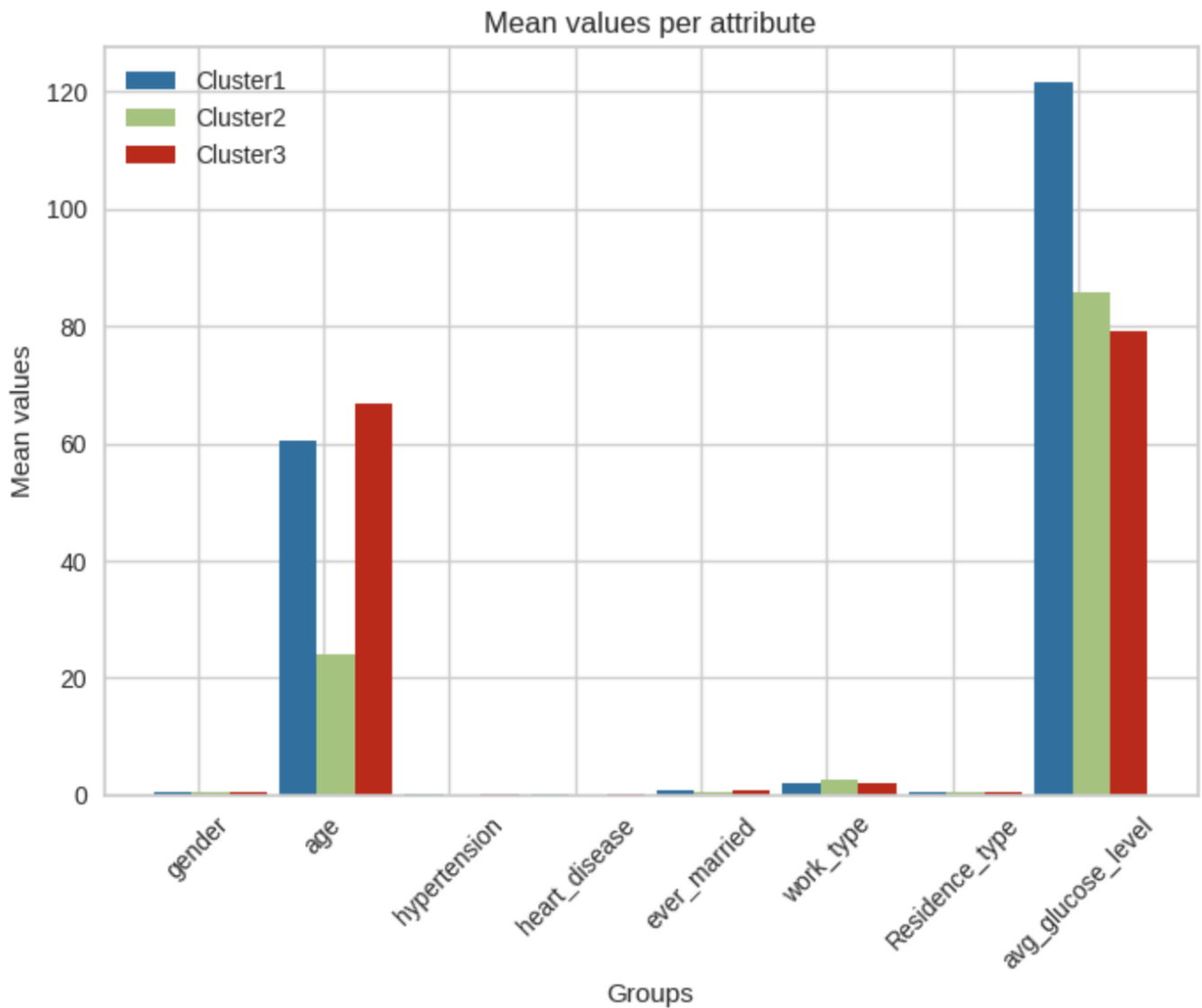
I started with 2 clusters for K-Means, and inferred the cluster centres. To identify the best number of clusters, I used Elbow to draw my visualizations (using the Calinski-Harabasz metric) as shown below with Yellowbrick.



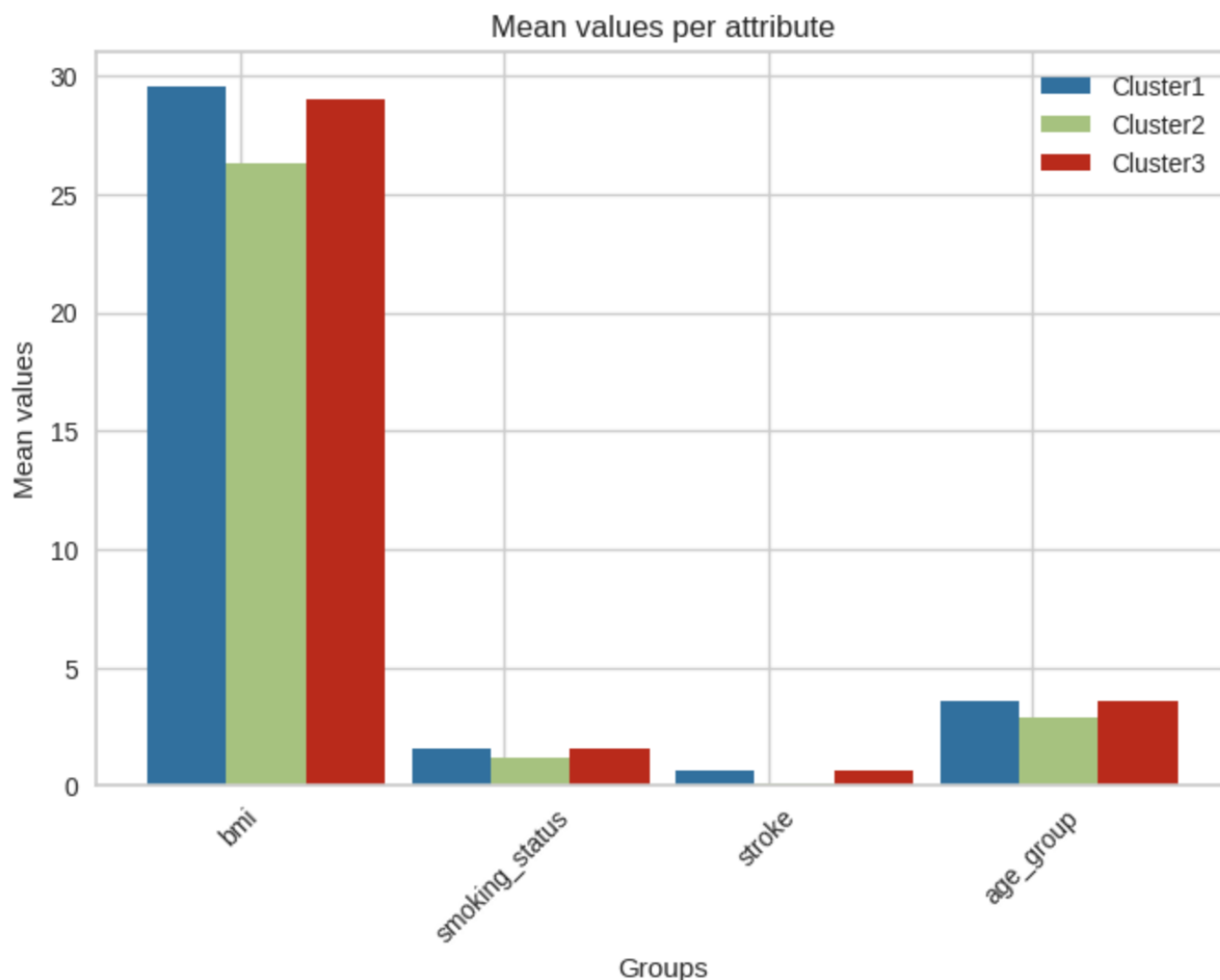
The plot shows the Calinski-Harabasz Score to identify the optimal number of clusters for K-Means clustering. The x-axis (k) shows the number of clusters chosen and the y-axis displays the Calinski-Harabasz score for the given clusters chosen. As we can see from the plot, the score is maximized at k=3; we should take to obtain optimal prediction clusters as 5935.526. However, the score drops off for k=4 and higher, which indicates a decrease in cluster performance. The K-Means model selection indeed benefits from this as three clusters is optimal for maximising cluster quality as per Calinski-Harabasz. Basically, this plot suggests that the optimal number of clusters must be 3, so data is fitted to k-means model with number of clusters to be 3.



From K-Means of a dataset with 8,232 samples from 9 clusters I computed the silhouette coefficient values for the clusters to determine how well a cluster is separated from the other clusters and plotted them. The average silhouette score (red dashed line) reflects the measure of clustering quality, where the higher silhouette score means better-defined and more separate clusters. The values of the silhouette coefficients are between -0.2 and +0.7 with most of the silhouette being positive which indicates us it's a reasonable separation between clusters. This means that some clusters have a lower or slightly negative coefficient, which shows overlap or bad separation. The plot depicts how well separate the clusters are from each other and how well grouped the clusters are to itself thus helping to ascertain if the selected number of clusters are a good fit.



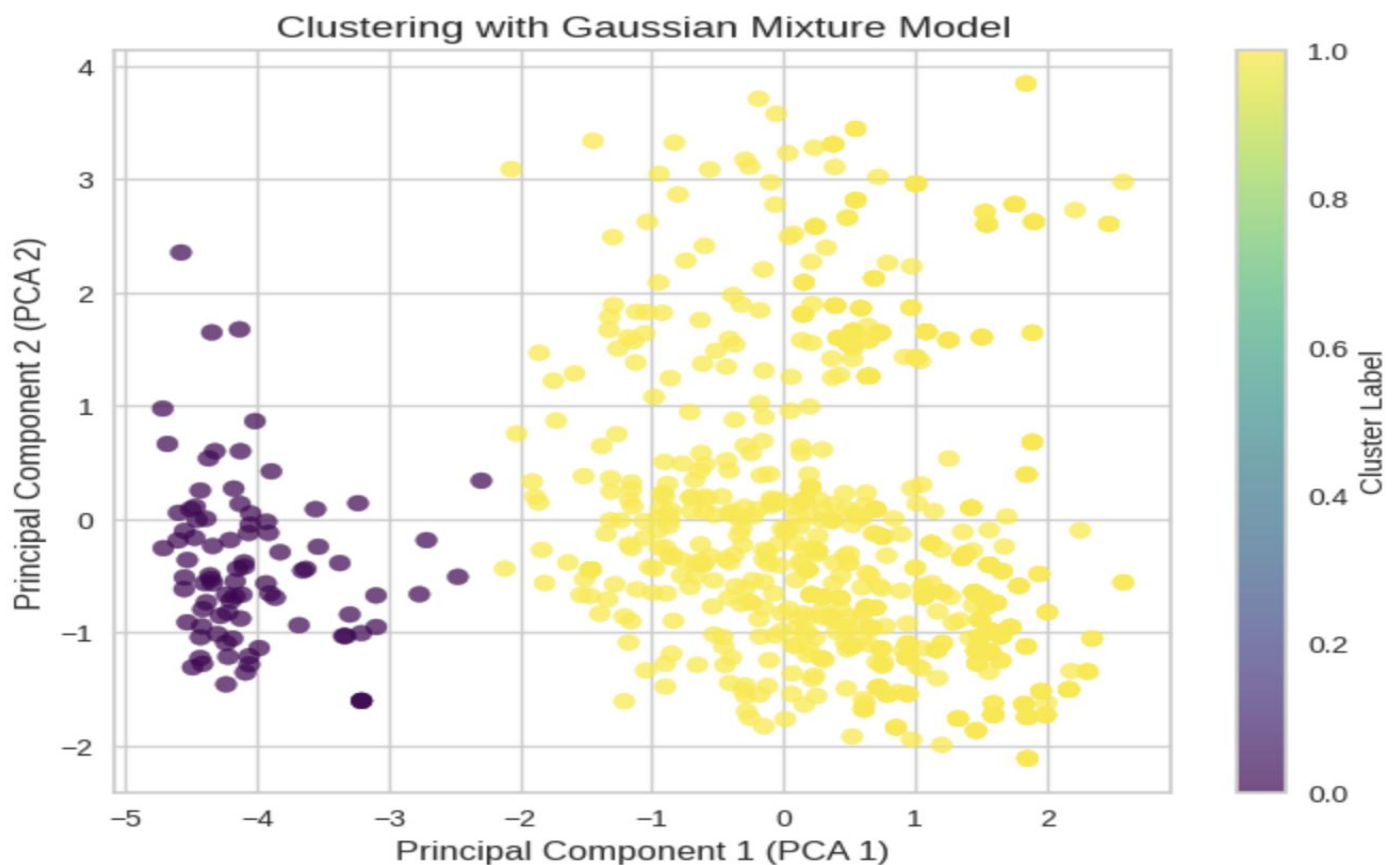
Above bar chart showing means of different attributes in 3 clusters (Cluster1, Cluster2, Cluster3). We analyze each attribute (i.e., gender, age, Hypertension, Heart-disease, Marital Status, then id Worker Type, Residence Type and Avg. Glucose level [mg/dl]) to its respective average contribution in every cluster. Interestingly, Cluster1 has the maximum average for the "avg_glucose_level," while age is maximum in cluster3. In relative terms, other attributes like hypertension and heart_disease reflect lower mean values in overall clusters leading to the presumption that these would not help offset between the clusters. Summary Two-dimensional projections provide some sense of how clusters differ according to mean values of the individual attributes within each cluster and can help in the interpretation of the clustering results.



The bar plot shows average values of four attributes (bmi: Body mass index, smoking: Smoking status, stroke: Whether the individual had a stroke, and age group: Age classification) across three clusters (cluster 1, cluster 2, and cluster 3). According to the data, BMI had the highest mean values in all clusters, with clusters one and three being nearly the same with slightly higher rates than cluster two. High p-values of smoking status and stroke show that there is little variation in these features across the clusters and potential mean values for any of the features are significantly low. Conversely, the age group attribute is not as consistent despite Clusters 1 and 2 having a better average than Cluster 3. These data underscore the predominance of BMI as a differentiating characteristic across the clusters, as well as the relative homogeneity of the smoking status and stroke characteristics.

b. Gaussian Mixture Model:

Clustering using GMM starts with importing corresponding libraries: GaussianMixture for clustering, PCA for dimensionality reduction, StandardScaler for feature scaling and matplotlib.pyplot for visualization. To reduce processing time while retaining some level of randomization, a sample comprising 10% of the entire dataset was taken. Features were standardized through StandardScaler to have zero mean and unit variance to improve GMM performance. identify probabilistic clusters A Gaussian Mixture Model (GMM) with 2 components & full covariance to predict the clusters. PCA was employed to condense the data into 2 dimensions for visualization of the high dimensional dataset. Finally, the clusters were plotted in a 2D space in form of a scatter plot where different colors clustered and indicated the cluster label displaying the output of the clustering process.



The scatter plot shows how the clusters found using the Gaussian Mixture Model (gmm) based on two principal components, PCA 1 and PCA 2. In Dark purple to yellow each colour of cluster label represents the predicted cluster probability level from each of the method The plot shows three main clusters — one formed on the left with negative PCA 1 values and two that overlap on the right with positive PCA 1 values. While separation along the PCA 1-axis is apparent in the distribution, PCA 2 provides further separation. This visualization calls attention to the fact that GMM can form probabilistic clusters with overlapping decision boundaries — especially in areas of space where the distribution of cluster entities is not uniform.

Comparison:

Both clustering techniques: K-means and GMM, were successful with the data. However, the number of clusters they found were quite different from each other. K-means cluster found three clusters while Gaussian Mixture Model predicted two clusters.

In terms of computational performance, both algorithms were quite fast and efficient, making them appropriate for dealing with this dataset. The difference in the number of clusters may reflect different underlying assumptions made by the two algorithms. Because K-means works under the assumption that nothing is inherently skewed and that clusters are spherical with equal variance chromosomes, K-means may split data into more than that of K-medoids segmentation. Gaussian Mixture Models, on the other hand, produces more fluid clusters with different shapes and sizes, which is probably why it found fewer clusters for the same data.

7.Reference:

- Scikit-learn developers. (n.d.). Scikit-learn: Machine learning in Python. Retrieved November 20, 2024, from <https://scikit-learn.org/stable/>
- University of Portsmouth, School of Computing. (2024). Programming for Data Analysis and AI (2024/25) Coursework (M33145). Dr. Atefeh Khazaei Ghoozhdi. Moodle. Retrieved December 29, 2024, from <https://moodle.port.ac.uk/course/view.php?id=25667>